

becoming-a-better-engineer

30-Day Coding Roadmap

June 15, 2026 → July 14, 2026

The Rules

- Stuck Rule: If stuck for 30+ minutes, write down why before asking for help.
- 70/30 Rule: 70% you, 30% AI. AI reviews, hints, explains — never writes the first solution.
- Miss a day? Next day you do double.

Daily Log Format (write this in NOTES.md before committing)

Day X — [Challenge Name]
Problem: (Rewrite in your own words)
My Logic: (How did you think about solving it?)
Pseudocode: (Steps in plain English first)
Challenges: (What did you get stuck on?)
Lessons Learned: (What do you know now that you didn't before?)

Week 1: Java Fundamentals

Goal: Stop second-guessing basic Java syntax and start writing code without fear.

Day 1 — Grade Calculator

Type: Exam / Beginner Java

Also appears as: Conditional logic in university exams and junior interviews

Problem:

Create a program that receives a student's score and returns a grade.

95 → A
82 → B
74 → C
63 → D
40 → F

Concepts:

if/else, methods, user input

Interview Angle:

"Write a method that takes a score and returns a grade." They're testing if you can structure a clean method and use conditions properly. A good answer also handles invalid input without crashing.

Exam Angle:

Very common in university Java exams. Expect variations like adding a method for class average, handling two students with the same score, or extending to A+/A/A- grades.

LeetCode Equivalent:

Not directly on LeetCode, but the logic pattern is the same as easy conditionals. Practice ground before harder problems.

Edge Cases to Think About:

- Score = 0
- Score = 100
- Score = -5
- Score = 101
- Score is a decimal (e.g. 84.5)

Day 2 — Attendance Checker

Type: Exam / Beginner Java

Also appears as: Boolean logic questions, eligibility systems

Problem:

Determine whether a student is eligible for an exam.

```
Attendance >= 75% → Eligible
```

```
Attendance < 75% → Not Eligible
```

Concepts:

conditions, booleans

Interview Angle:

Simple on the surface, but interviewers look at how you handle the boundary (exactly 75%). Do you use `>=` or `>`? That detail matters and shows precision.

Exam Angle:

Often extended to include multiple conditions — e.g. attendance AND assignment submission AND no outstanding fees.

LeetCode Equivalent:

Similar boolean logic to LC #1790 (Check if One String Swap Can Make Strings Equal) in terms of true/false reasoning.

Edge Cases to Think About:

- Attendance = 75 exactly
- Attendance = 0
- Attendance = 100

- Attendance = 74.9

Day 3 — ATM Menu

Type: Exam / Beginner Java

Also appears as: Menu-driven programs, switch statement questions

Problem:

Allow a user to: Check Balance, Deposit, or Withdraw.

1. Check Balance
2. Deposit
3. Withdraw

Concepts:

switch, methods, loops

Interview Angle:

Tests whether you can structure a program with a menu loop. Interviewers look for: does the menu keep running after each action? A real ATM doesn't close after one transaction.

Exam Angle:

Common in first-year Java exams. Variations include adding a PIN check, limiting withdrawals, or tracking transaction history.

LeetCode Equivalent:

State management logic similar to LC #362 (Design Hit Counter) — maintaining and updating a value over time.

Edge Cases to Think About:

- Withdraw more than balance
- Deposit a negative amount
- Enter an invalid menu option
- Balance = 0 and user tries to withdraw

Day 4 — Simple Calculator

Type: Exam / Beginner Java

Also appears as: Method design questions, edge case handling

Problem:

Build a calculator that handles add, subtract, multiply, and divide.

```
10 + 5 → 15
10 / 0 → "Cannot divide by zero"
```

Concepts:

methods, conditions, edge cases

Interview Angle:

This is a classic first interview question. They're not testing the math — they're testing if you remember to handle division by zero. Missing that is an instant red flag.

Exam Angle:

Exams often ask you to extend this: add modulo support, handle decimal results, or make it loop for multiple calculations.

LeetCode Equivalent:

LC #150 (Evaluate Reverse Polish Notation) — same operators, more complex structure. Day 4 is your foundation for that.

Edge Cases to Think About:

- Division by zero
- Negative numbers
- Decimal results
- Very large numbers

Day 5 — Student Average Calculator

Type: Exam / Beginner Java

Also appears as: Array + loop questions, aggregate calculation

Problem:

Input marks for 5 subjects. Calculate the average and return the grade.

Concepts:

arrays, loops

Interview Angle:

Tests if you can iterate over a collection and compute something. Interviewers sometimes extend it: "Now find the highest mark" or "Remove the lowest mark before averaging."

Exam Angle:

Extremely common. Variations include weighted averages, finding min/max, and sorting the marks.

LeetCode Equivalent:

LC #1491 (Average Salary Excluding the Minimum and Maximum Salary) — same pattern with a twist.

Edge Cases to Think About:

- All marks are 0
- One subject mark is missing
- Mark is above 100
- All marks are the same

Day 6 — Library Book Tracker

Type: Exam / Beginner Java

Also appears as: Array management, basic CRUD without a database

Problem:

Store book titles in an array. Display all books. Allow adding a new book.

Concepts:

arrays, loops, basic CRUD thinking

Interview Angle:

This is your first taste of thinking like a backend developer. Can you store data, retrieve it, and display it? Interviewers may ask: "How would you search for a book by title?"

Exam Angle:

Often extended to include searching, deleting, or counting available vs checked-out books.

LeetCode Equivalent:

Logic foundation for LC #706 (Design HashMap) — storing and retrieving structured data.

Edge Cases to Think About:

- Array is full and user tries to add another book
- Book title is empty
- Searching for a book that doesn't exist
- Duplicate book titles

Day 7 — Review Day

Type: Retention check

Problem:

Rebuild Days 1–6 from memory. No tutorials. No AI. No looking at your old code first. If you can't remember something, write it in NOTES.md — that's what you need to study.

Week 2: Problem Solving (JavaScript)

Goal: Learn to break a problem down before writing a single line of code.

Day 8 — Count Vowels

Type: String manipulation / LeetCode Easy

Also appears as: Coding tests, screening rounds

Problem:

Count the number of vowels in a string.

Input: "Rwanda"

Output: 2

Concepts:

strings, loops, conditions

Interview Angle:

Often used as a warm-up in phone screens. They want to see how you approach it — do you loop through, use a Set for vowels, handle uppercase?

Exam Angle:

Variations include counting consonants, counting specific characters, or returning the vowels themselves.

LeetCode Equivalent:

Foundation for LC #1704 (Determine if String Halves Are Alike).

Edge Cases to Think About:

- Empty string
- String with no vowels
- Uppercase vowels (AEIOU)
- String with numbers or symbols

Day 9 — Reverse a String

Type: String manipulation / LeetCode Easy

Also appears as: Almost every beginner coding test in existence

Problem:

Reverse a string without using the built-in reverse method.

Input: "hello"

Output: "olleh"

Concepts:

strings, loops or two-pointer approach

Interview Angle:

Interviewers often say "don't use the built-in reverse method." Know how to do it manually with a loop or two-pointer approach. That separates someone who understands it from someone who Googled it.

Exam Angle:

Extended to: reverse only the words in a sentence, reverse only vowels, or check if a string is a palindrome.

LeetCode Equivalent:

LC #344 (Reverse String) — exact same problem.

Edge Cases to Think About:

- Empty string
- Single character
- String with spaces
- String with special characters

Day 10 — Find Largest Number

Type: Array traversal / LeetCode Easy

Also appears as: University exams, screening tests

Problem:

Find the largest number in an array without using Math.max().

Input: [10, 20, 5, 90]

Output: 90

Concepts:

arrays, loops, comparison

Interview Angle:

Try solving without Math.max(). Then they'll ask: "Now find the second largest." That's the real test.

Exam Angle:

Variations: find the smallest, find both min and max, find the index of the largest number.

LeetCode Equivalent:

Foundation for LC #215 (Kth Largest Element in an Array).

Edge Cases to Think About:

- Array with one element
- All elements are the same
- Array with negative numbers
- Empty array

Day 11 — Remove Duplicates

Type: Array manipulation / LeetCode Easy-Medium

Also appears as: Interviews, data cleaning problems

Problem:

Remove duplicate values from an array.

Input: [1, 1, 2, 2, 3, 3]

Output: [1, 2, 3]

Concepts:

arrays, Set, loops

Interview Angle:

There are multiple ways — using a Set, filter, or a loop with includes(). Interviewers care which you choose and why. Using a Set shows you know your data structures.

Exam Angle:

Variations: remove duplicates and keep only the duplicated ones, count duplicates, return duplicates instead of unique values.

LeetCode Equivalent:

LC #26 (Remove Duplicates from Sorted Array) and LC #217 (Contains Duplicate).

Edge Cases to Think About:

- Array with no duplicates
- Array where everything is a duplicate
- Empty array
- Array with one element

Day 12 — Character Frequency Counter

Type: Hash map / LeetCode Easy-Medium

Also appears as: Interviews constantly — this pattern shows up everywhere

Problem:

Count how many times each character appears in a string.

Input: "banana"

Output: { b:1, a:3, n:2 }

Concepts:

objects, loops, frequency patterns

Interview Angle:

One of the most important patterns in coding interviews. Once you understand frequency counting, you can solve dozens of other problems. Tests if you know when to reach for a hash map.

Exam Angle:

Variations: find the most frequent character, find the first non-repeating character, check if two strings are anagrams.

LeetCode Equivalent:

LC #387 (First Unique Character in a String), LC #242 (Valid Anagram) — all built on this same pattern.

Edge Cases to Think About:

- Empty string
- String with spaces
- Case sensitivity (is 'A' the same as 'a?')

- String with numbers

Day 13 — Palindrome Checker

Type: String manipulation / LeetCode Easy

Also appears as: Classic interview question, university exams

Problem:

Check if a string reads the same forwards and backwards.

```
madam → true
hello → false
```

Concepts:

strings, two-pointer logic, reverse comparison

Interview Angle:

Solve it two ways: by reversing and comparing, AND using the two-pointer approach. Knowing both shows depth. The two-pointer approach is more efficient and impresses interviewers.

Exam Angle:

Extended to: ignore spaces and punctuation, check if a number is a palindrome, find all palindromic substrings.

LeetCode Equivalent:

LC #125 (Valid Palindrome) — same problem with added constraints.

Edge Cases to Think About:

- Empty string
- Single character (always a palindrome)
- String with spaces ("race car")
- Case sensitivity ("Madam")
- Numbers as strings ("121")

Day 14 — Review Day

Type: Retention check

Problem:

Rebuild your 3 favorite solutions from Days 8–13 from memory. Then pick the hardest one and try to solve it a different way.

Week 3: React

Goal: Build a component from scratch without copying. Just you and the React docs.

Day 15 — Counter App

Type: React basics / Junior frontend interview

Also appears as: First React question in almost every junior interview

Problem:

Build a counter with Increase, Decrease, and Reset buttons.

Concepts:

useState, event handlers, conditional rendering

Interview Angle:

Interviewers use this to check if you understand useState. They'll extend it: "Add a max limit of 10" or "Disable the Decrease button when count is 0."

Exam Angle:

Extensions: step counter (increase by 5), multiple counters, persisting the count.

Edge Cases to Think About:

- Count goes negative — should it be allowed?
- What happens at 0 when user clicks Decrease?
- Reset from a very large number

Day 16 — Todo App

Type: React CRUD / Junior-Mid frontend interview

Also appears as: The most common React project in interviews and portfolios

Problem:

Add a task, delete a task, and mark a task as complete.

Concepts:

useState, lists, .map(), .filter(), events

Interview Angle:

The React interview. They'll ask: "How would you persist this data?" (localStorage or API). "How would you edit a task?" Know the answer even if you don't implement it.

Exam Angle:

Extensions: edit a task, filter by complete/incomplete, count remaining tasks.

Edge Cases to Think About:

- Adding an empty task
- Deleting the only task
- Very long task name
- Adding 100+ tasks (performance?)

Day 17 — Student List

Type: React props and lists / Junior interview

Also appears as: Data rendering questions

Problem:

Display a list of students from an array using `.map()`. Each student has a name, age, and grade.

Concepts:

props, `.map()`, component structure

Interview Angle:

Tests if you understand component decomposition. Do you put everything in one component or create a `StudentCard` component? The second approach is the right answer.

Exam Angle:

Extensions: sort by name, sort by grade, highlight failing students in red.

Edge Cases to Think About:

- Empty student list
- Student with missing data (no grade yet)
- Very long name

Day 18 — Search Filter

Type: React controlled inputs / Junior-Mid interview

Also appears as: Live search, filtering UI

Problem:

Search students by name in real time as the user types.

Concepts:

`useState`, controlled inputs, `.filter()`

Interview Angle:

Tests controlled components. Interviewers look for: do you filter on every keystroke, handle case sensitivity, show a 'no results' message when nothing matches?

Exam Angle:

Extensions: filter by grade, filter by multiple fields, debounce the search input.

Edge Cases to Think About:

- Search term with no matches
- Searching with uppercase vs lowercase
- Empty search (should show all students)
- Search with spaces

Day 19 — Expense Tracker

Type: React state + forms / Junior-Mid interview

Also appears as: Budget apps, finance UI questions

Problem:

Add expense name and amount. Display all expenses and a running total.

Concepts:

useState, forms, array of objects, computed values

Interview Angle:

Tests if you can manage an array of objects in state. Interviewers look at how you calculate the total — do you recalculate on every render properly? Do you validate the amount input?

Exam Angle:

Extensions: delete an expense, categorize expenses, show highest expense.

Edge Cases to Think About:

- Amount is 0
- Amount is negative
- Expense name is empty
- Very large amount

Day 20 — Theme Toggle

Type: React context or state / Junior-Mid interview

Also appears as: Dark mode implementation, UI preference questions

Problem:

Toggle between Light Mode and Dark Mode.

Concepts:

useState, conditional className, CSS variables or inline styles

Interview Angle:

Simple version uses useState. Interviewers may ask "how would you make this available across the whole app?" — the answer is React Context. Know that even if you don't implement it today.

Exam Angle:

Extensions: remember preference with localStorage, add a third theme, apply theme to nested components.

Edge Cases to Think About:

- Default theme on first load
- Flash of wrong theme on page reload

- Applying theme consistently to all elements

Day 21 — Review Day

Type: Retention check

Problem:

Rebuild Counter, Todo, and Search Filter without tutorials or your old code. Focus on: can you set up useState correctly from scratch without looking it up?

Week 4: Backend + Testing Mindset

Goal: Think about what can break, not just what should work.

Day 22 — Design Test Cases

Type: QA / Testing mindset / Junior QA or dev interview

Also appears as: Manual testing questions, ISTQB concepts

Problem:

Feature: Login Form. Write positive tests, negative tests, and edge cases. No code — just think like a tester.

Concepts:

test case design, equivalence partitioning, boundary value analysis

Interview Angle:

Very common in QA interviews and increasingly in dev interviews. A strong answer covers: valid login, wrong password, empty fields, SQL injection attempt, account locked after 3 tries.

Exam Angle:

You'll be given a feature and asked to write 10–15 test cases. Structure matters: Test ID, Description, Input, Expected Output, Pass/Fail.

Edge Cases to Think About:

- Correct email, wrong password
- Wrong email, correct password
- Both fields empty
- Email with spaces
- What happens after 5 failed attempts?
- Copy-paste into password field

Day 23 — Bug Hunting

Type: QA thinking / Dev problem-solving

Also appears as: Code review, QA interviews, debugging exercises

Problem:

Find 10 possible bugs in an e-commerce checkout flow (adding to cart → payment → confirmation).

Concepts:

QA mindset, user flow thinking, edge cases

Interview Angle:

Interviewers give you a feature or user story and ask what could go wrong. They're testing whether you think like an engineer who ships reliable software, not just software that works in ideal conditions.

Exam Angle:

You may be given a short piece of code or a feature description and asked to list defects or risks.

Edge Cases to Think About:

- What if payment fails halfway through?
- What if the item goes out of stock during checkout?
- What if the user double-clicks the Pay button?
- What if the discount code is applied twice?
- What if the user's session expires during payment?

Day 24 — REST API Design

Type: Backend / System design / Junior-Mid backend interview

Also appears as: API design questions, system design rounds

Problem:

Design endpoints for a Student Management System.

GET	/students	→ get all students
POST	/students	→ create a student
GET	/students/:id	→ get one student
PUT	/students/:id	→ update a student
DELETE	/students/:id	→ delete a student

Concepts:

REST principles, HTTP methods, resource naming

Interview Angle:

They'll ask: "What's the difference between PUT and PATCH?" (PUT replaces the whole object, PATCH updates only changed fields). Know your HTTP status codes: 200, 201, 400, 401, 404, 500.

Exam Angle:

Design the API for a library system, hospital system, or banking system. Same pattern, different domain.

Edge Cases to Think About:

- GET /students when there are no students → return [] not an error
- POST with missing required fields → 400 Bad Request
- DELETE a student that doesn't exist → 404
- Two requests try to create the same student simultaneously

Day 25 — Validation Logic

Type: Backend logic / Interview

Also appears as: Form validation, API input validation

Problem:

Write validation for Email, Password, and Phone Number.

Email: must contain @, valid domain

Password: min 8 chars, 1 uppercase, 1 number, 1 special character

Phone: 10 digits, handle +250 format (Rwanda)

Concepts:

regex, conditions, error messages

Interview Angle:

They're testing if you think about security and data integrity. Always return a clear error message, not just true/false.

Exam Angle:

You may be asked to write the regex from scratch or explain what a given regex does.

Edge Cases to Think About:

- Email with multiple @ symbols
- Password that's exactly 8 characters
- Phone number with spaces or dashes
- International phone numbers

Day 26 — Build a Mini REST API

Type: Backend development / Junior backend interview

Also appears as: Take-home projects, coding challenges

Problem:

Build an API: POST /students, GET /students, DELETE /students/:id. Use Java (Spring Boot) or Node.js (Express).

Concepts:

routing, request/response, in-memory data storage

Interview Angle:

Junior backend interviews often end with "build a simple CRUD API." They look at: does it run, does it handle errors, does it return correct status codes, is the code readable?

Exam Angle:

You'll be given a domain (books, employees, products) and asked to build the same structure.

Edge Cases to Think About:

- DELETE on an ID that doesn't exist
- POST with no body
- GET when the list is empty
- Two students with the same name

Day 27 — Write Unit Tests

Type: Testing / Junior dev interview

Also appears as: TDD questions, Jest/JUnit exercises

Problem:

Write unit tests for `calculateGrade()` from Day 1. Test normal inputs, boundary values, and invalid inputs.

```
Arrange → set up the input
Act      → call the function
Assert   → check the output
```

Concepts:

unit testing, assertions, Arrange-Act-Assert structure

Interview Angle:

More companies expect junior devs to write tests. They look for: do you test the happy path, edge cases, and error cases? Do you know the difference between a unit test and an integration test?

Exam Angle:

You'll be given a function and asked to write tests that cover all scenarios.

Edge Cases to Think About:

- `calculateGrade(95)` → A
- `calculateGrade(50)` → F
- `calculateGrade(75)` → boundary — what grade is this?
- `calculateGrade(-1)` → should throw an error or return null
- `calculateGrade(101)` → invalid input handling

Day 28 — Refactor Day

Type: Code quality / Mid-level interview skill

Also appears as: Code review, clean code exercises

Problem:

Pick any code from Week 1 or Week 2. Improve naming, remove duplicate logic, break long methods into smaller ones, and add comments where logic isn't obvious.

Concepts:

clean code, readability, maintainability

Interview Angle:

Senior devs judge juniors by how readable their code is. 'Does it work' is the minimum. 'Is it easy to understand 3 months later' is the real bar.

Edge Cases to Think About:

- Would someone who didn't write this understand it in 5 minutes?
- Are there methods doing more than one thing?
- Are there magic numbers that should be named constants?
- Is any logic repeated that could be a helper method?

Day 29 — Final Project

Type: Portfolio project / Interview talking point

Also appears as: Take-home assignment, junior portfolio piece

Problem:

Build a Student Management System: Add student, view all students, search by name, validate all input, write at least 5 unit tests.

Concepts:

Everything from Weeks 1–4 combined

Interview Angle:

This is what you talk about in interviews when they ask 'show me something you built.'

Day 30 — Reflection

Type: Personal review

Problem:

No code today. Write honestly in NOTES.md: - What was the hardest day and why? - What do you understand now that you didn't on Day 1? - Which week felt most natural? - Which week do you need to revisit? - What do you want to build next?